HOSPEDJÁ

CHECKPOINT 3

Projetos de Software 2 • UNIUBE

CodeSync - Grupo 3 | Equipe de 5 integrantes

Relatório técnico consolidado • 24 de outubro de 2026

Versão acadêmica para revisão. Testes funcionais, cobertura real e deploy AWS permanecem pendentes de comprovação.

Material de apoio: código-fonte, SQL, OpenAPI, diagramas editáveis e documentação no ZIP/repositório.

1. Escopo, objetivo e situação da entrega

O HospedJá é uma plataforma web de pesquisa, comparação e reserva de hospedagens, com guias turísticos e atendimento SAC/Ouvidoria. Esta etapa concentra-se na implementação inicial de serviços REST e organização dos artefatos do Checkpoint 3, preservando os requisitos do Checkpoint 2.

Escopo acadêmico conhecido

• 4.8: definir tecnologia backend, desenvolvimento orientado por testes (TDD), implementação REST e implantação em nuvem.

• 4.9: diagramas de classes das principais interações entre os componentes.

• 4.9.2: atualização semanal no ProjectLibre - não produzida neste pacote, conforme solicitação da equipe; verificar exigência docente.

• 4.9.3: commits semanais no GitHub - apenas instruções e workflow disponíveis; histórico real cabe ao grupo.

• Documento técnico e artefatos em PDF único e apresentação em grupo.

Status honesto

Item	Situação na consolidação
Banco	DDL e seed produzidos; execução em MySQL 8 não comprovada
API Express	Código organizado em camadas e endpoints presentes; testes runtime não comprovados
TDD / Jest	Testes e laboratório didático escritos; resultados e cobertura reais pendentes
Swagger	OpenAPI em arquivo; carregamento real pendente
AWS	Plano EC2/RDS e scripts; não implantado nem URL pública
Diagramas	6 arquivos `.drawio` editáveis e visualizações neste documento

Origem dos requisitos

O documento técnico do CP2 enumera RF01–RF35, RNF01–RNF14 e as regras RN01–RN06. Logs são RF32, RF33 e RF34; RF26 refere-se a respostas automatizadas da IA. Esta é a referência adotada na matriz de rastreabilidade.

2. Arquitetura em camadas

Stack adotada: Node.js 22 + Express.js, MySQL 8, mysql2, Zod, bcrypt, JWT, Winston, Jest, Supertest, Swagger/OpenAPI, Docker e proposta de AWS EC2 + RDS.

Camada	Responsabilidades
Routes	Mapeamento dos endpoints e encadeamento de middlewares HTTP
Controllers	Recepção de parâmetros, respostas e códigos HTTP
Services	Casos de uso e validação de permissões no fluxo de domínio
Repositories	Consultas SQL parametrizadas e acesso às entidades
Database	Persistência transacional MySQL 8 e tabela de auditoria

Contratos e integrações

A API responde JSON padronizado, usa Bearer JWT em rotas privadas e documentação em /api-docs/. Mercado Pago e DeepSeek estão previstos no desenho de domínio, mas não foram integrados. O diagrama legado Java/Python foi substituído por uma visão Node.js/Express compatível com o código reunido.

Restrições deliberadas desta etapa

• Reservas são rascunhos PENDENTE; não calculam preço, não selecionam quarto nem confirmam disponibilidade.

• Pagamentos criados pela API são somente registros administrativos preliminares; não aprovam transações nem efetuam estorno.

• Ocorrências podem ser registradas e analisadas por ADMIN; não se aplica punição automaticamente.

3. Banco de dados e entidades

O SQL consolidado contém 40 instruções CREATE TABLE IF NOT EXISTS no arquivo database/01_schema.sql, com chaves primárias e estrangeiras, índices e restrições. O script database/02_seed.sql cria apenas dados fictícios para desenvolvimento; database/03_verificacao.sql reúne consultas de checagem.

Agrupamento	Tabelas/entidades representativas
Identidade e cadastro	tb_usuario, tb_endereco_usuario, tb_admin_hotel
Catálogo de hospedagens	tb_hotel, tb_endereco, tb_tipo_quarto, imagens e mídia
Reserva e pagamento	tb_reserva, tb_item_reserva, tb_pagamento, status e históricos
Guias	tb_guia, tb_reserva_guia, avaliações
SAC e comunicação	tb_atendimento, tb_mensagem, tb_atendimento_historico
Ocorrências	tb_ocorrencia, tb_penalidade
Auditoria	tb_log e logs setoriais

Cuidados com consistência

• O modelo mapeia a classe Hospedagem como tb_hotel e quartos como tb_tipo_quarto.

• Relações e FKs devem ser testadas em um MySQL 8 real; contagens estáticas não substituem execução.

• Não enviar o seed fictício para RDS/produção. Senhas de exemplo possuem hash bcrypt, mas são apenas contas de laboratório.

4. API REST e autenticação

Área	Endpoints principais / situação
Autenticação	POST /auth/register, POST /auth/login, PATCH /auth/password
Usuários	GET/PATCH /usuarios/me; administração /usuarios
Catálogo	GET/POST/PATCH/DELETE em /hoteis, /quartos, /guias
Reservas	GET/POST/PATCH/DELETE /reservas - somente rascunhos
Pagamentos	GET/POST /pagamentos - registro interno preliminar
Ocorrências	GET/POST /ocorrencias, PATCH /ocorrencias/:id/analise

Segurança e validação

• Cadastro aplica bcrypt no servidor; login emite JWT; middleware verifica token e consulta papel/status atuais no banco.

• Zod valida corpos e parâmetros. SQL usa parâmetros em consultas. Rotas administrativas checam perfil ADMIN ou vínculo do gestor.

• Winston e tb_log recebem eventos mutantes; evitar senha/token em logs. Testar se há registro persistido, pois erro de auditoria não cancela a operação.

Documentação executável

A especificação OpenAPI está em src/docs/openapi.yaml. Após subir a aplicação, abrir /api-docs/ e /openapi.json. A compatibilidade precisa ser verificada contra respostas reais.

5. Testes, TDD e cobertura

A base inclui testes unitários, HTTP com Supertest, integração opcional com banco isolado, pipeline CI e laboratório didático dos ciclos Red, Green e Refactor. O laboratório foi preparado depois do código inicial, portanto não comprova histórico test-first nas Partes 1–2.

Comando	Objetivo
npm run check	Parse/sintaxe de arquivos JS
npm run integrity	Referências entre módulos e estrutura SQL
npm test	Testes unitários e HTTP simulados
npm run test:coverage	Relatório Jest e cobertura em coverage/
npm run test:db	Integração MySQL - requer banco de teste separado
npm run test:tdd:red	Cenário Red didático; falhas esperadas
npm run test:tdd:green	Implementação mínima de referência
npm run test:tdd:refactor	Refatoração do laboratório

Evidência necessária, ainda não fornecida

• Relatório real do Jest, cobertura medida, health consultando MySQL, Swagger aberto e auditoria sem segredos.

• GitHub Actions verde e PRs que mostrem testes antes de novos recursos. Capturas devem omitir credenciais.

6. Implantação AWS - proposta, não executada

Proposta de baixo custo relativa: EC2 pequena executando a API em container, RDS MySQL 8 em rede privada, HTTPS via Nginx/Certbot, segredos no Systems Manager Parameter Store e backups. A viabilidade do preço, Free Tier e os recursos são condicionados à conta e à região; consultar orçamento atual antes de provisionar.

Fluxo de implantação

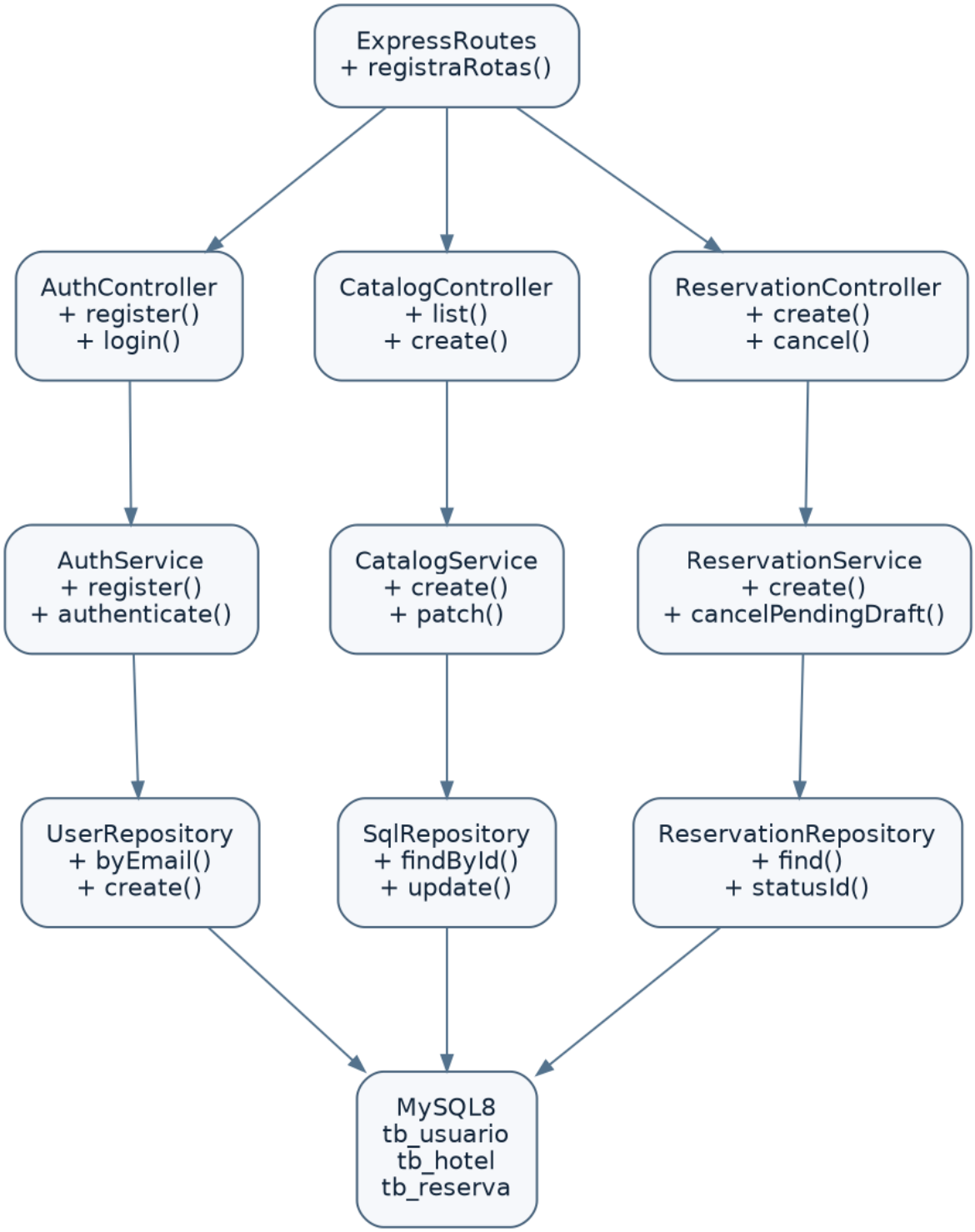
- Configurar orçamento, IAM e MFA no console AWS.
- Provisionar RDS privado com Security Group permitindo MySQL apenas a partir da EC2.
- Importar schema SQL; usar script de status básicos, não o seed de contas fictícias.
- Provisionar EC2, instalar Docker, AWS CLI, Nginx e certificados.
- Popular Parameter Store com segredos, sem incluí-los no GitHub.
- Executar aws/deploy-ec2.sh, configurar proxy HTTPS e verificar /health e Swagger.
- Produzir prints reais: HTTPS público, RDS privado, audit trail, resultados de testes e orçamento.

Riscos de implantação

A inexistência de uma URL pública verificada significa que o critério de implantação em nuvem não pode ser marcado como atendido. Consultar aws/DEPLOY_AWS.md para os procedimentos e eventuais custos orientativos.

7. Diagrama: arquitetura em camadas

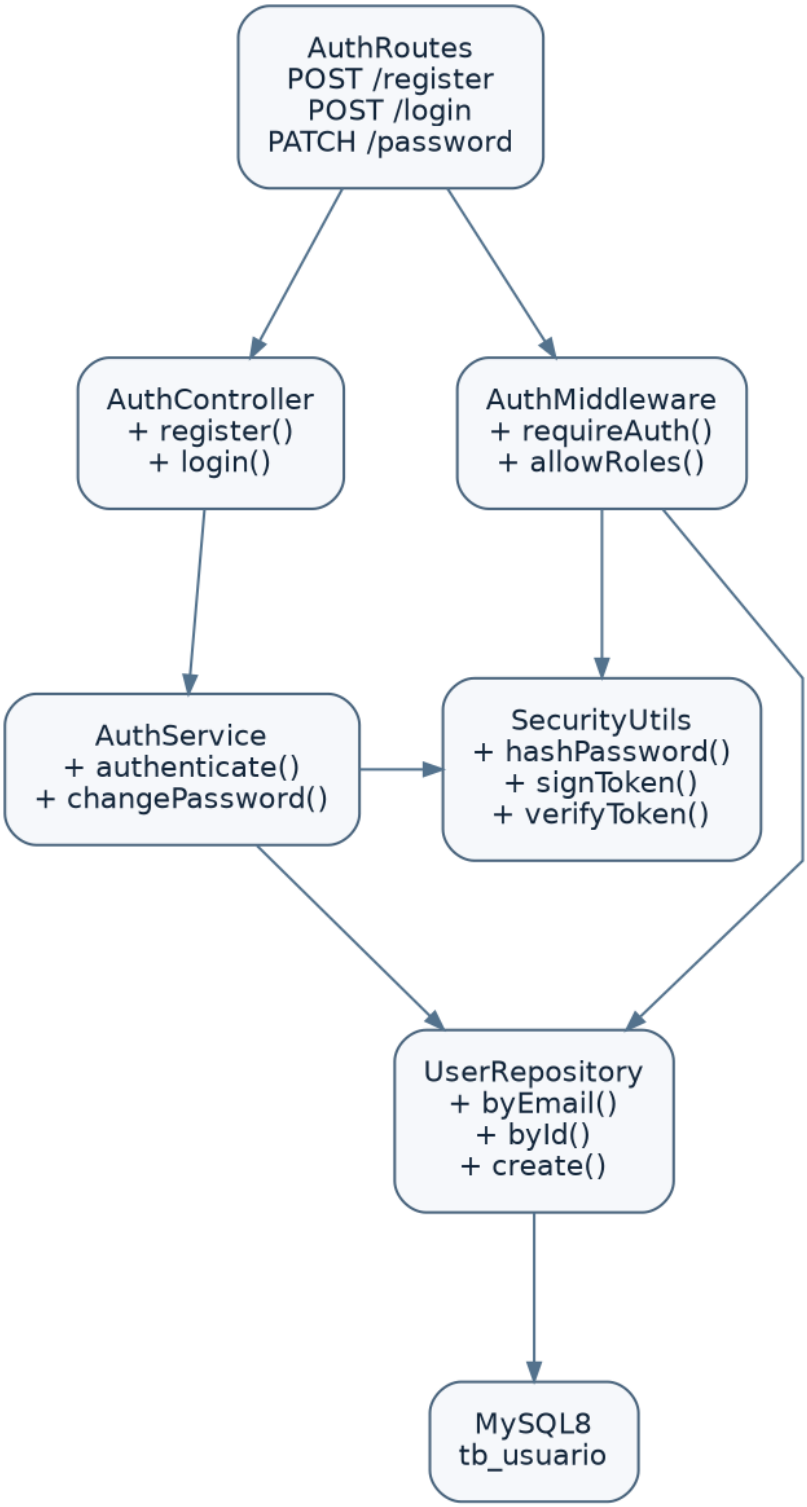
Principais dependências de rotas, controladores, serviços e repositórios no backend.



Fonte: diagrama de classes de interação editável no arquivo diagramas/01_camadas_controller_service_repository.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

8. Diagrama: autenticação JWT e bcrypt

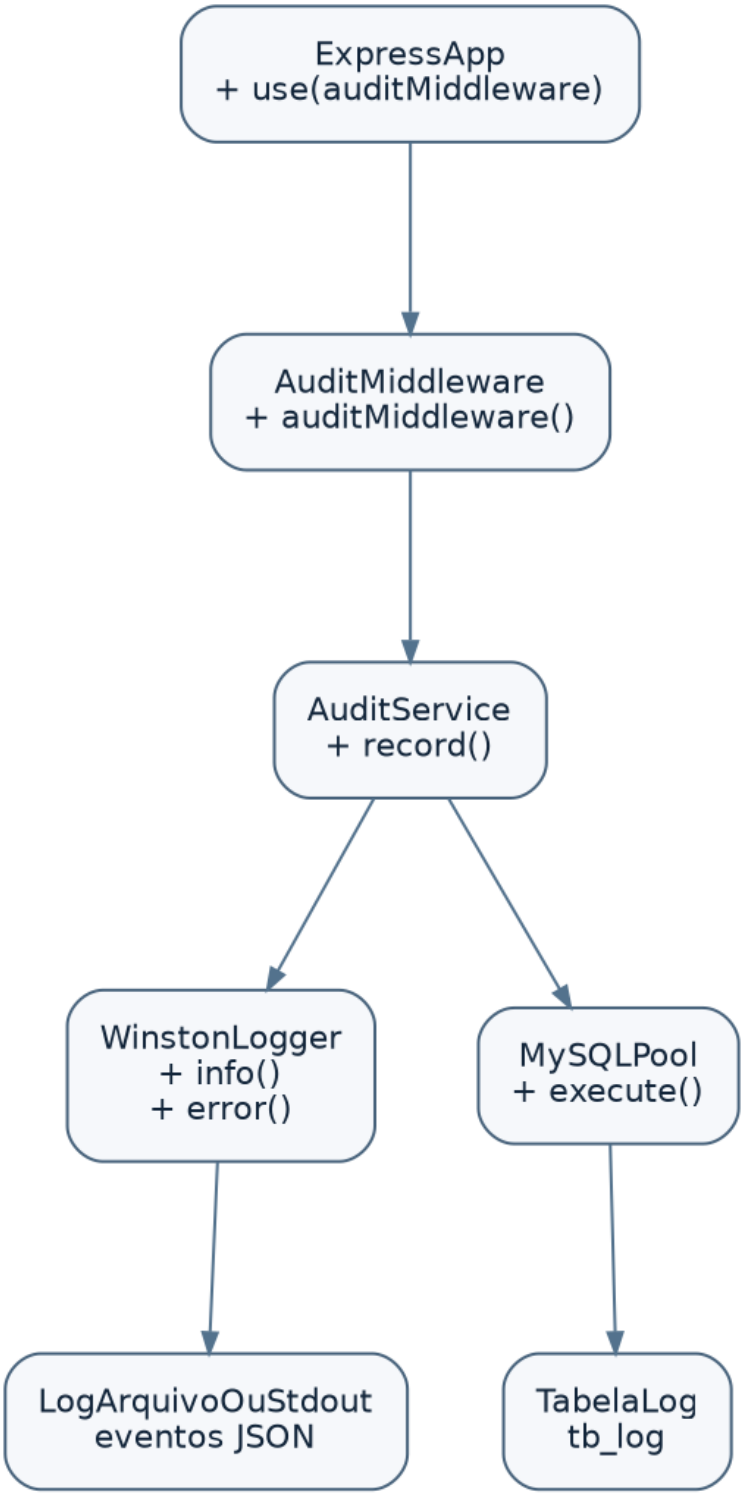
Fluxo de autenticação e validação de identidade/perfil; o banco permanece fonte de verdade dos papéis.



Fonte: diagrama de classes de interação editável no arquivo diagramas/02_autenticacao_jwt_bcrypt.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

9. Diagrama: auditoria e logs

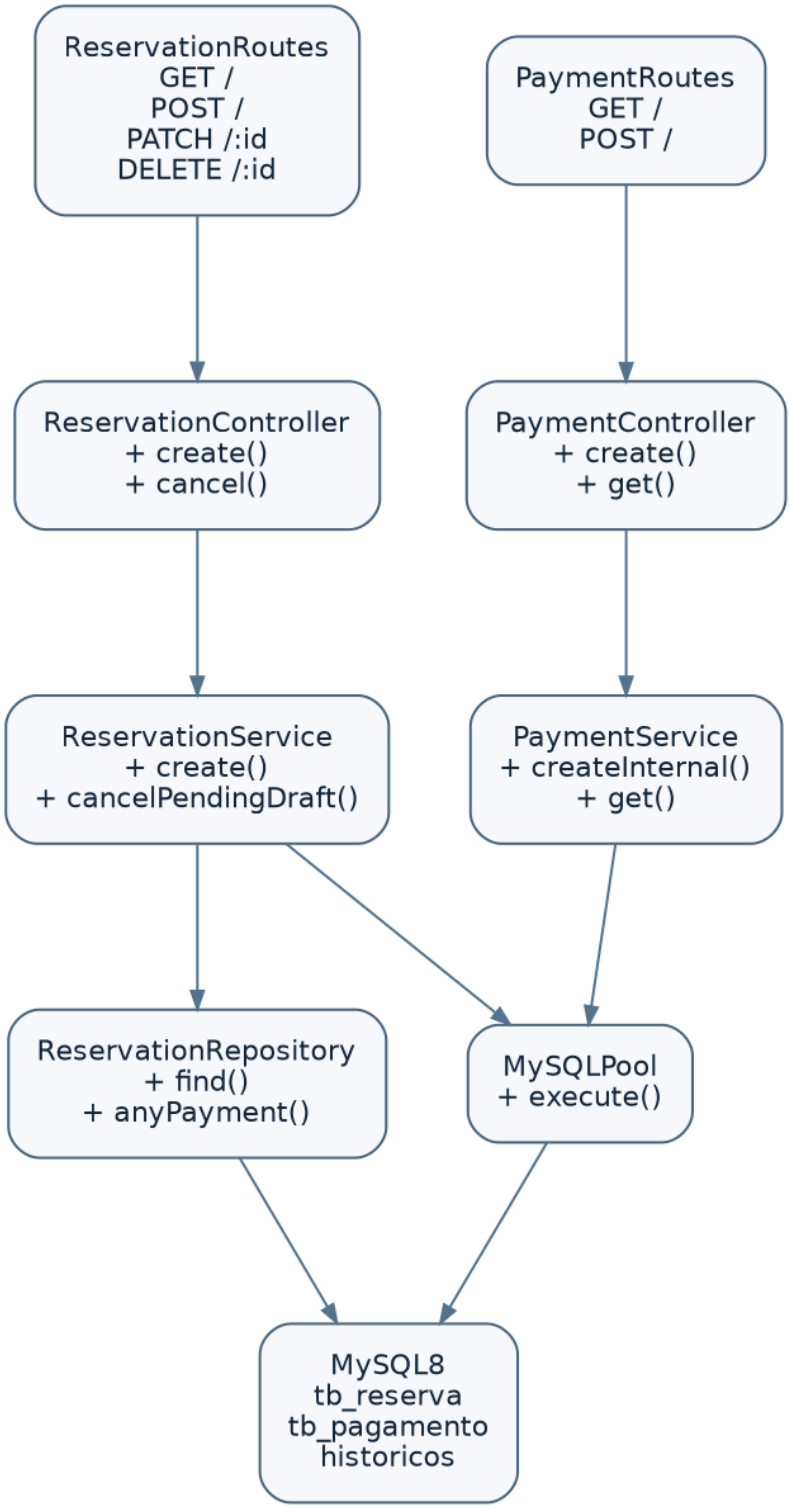
Registro transversal de eventos mutantes em Winston e banco, com limitações de persistência explicitadas.



Fonte: diagrama de classes de interação editável no arquivo diagramas/03_auditoria_logs.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

10. Diagrama: reservas e pagamentos

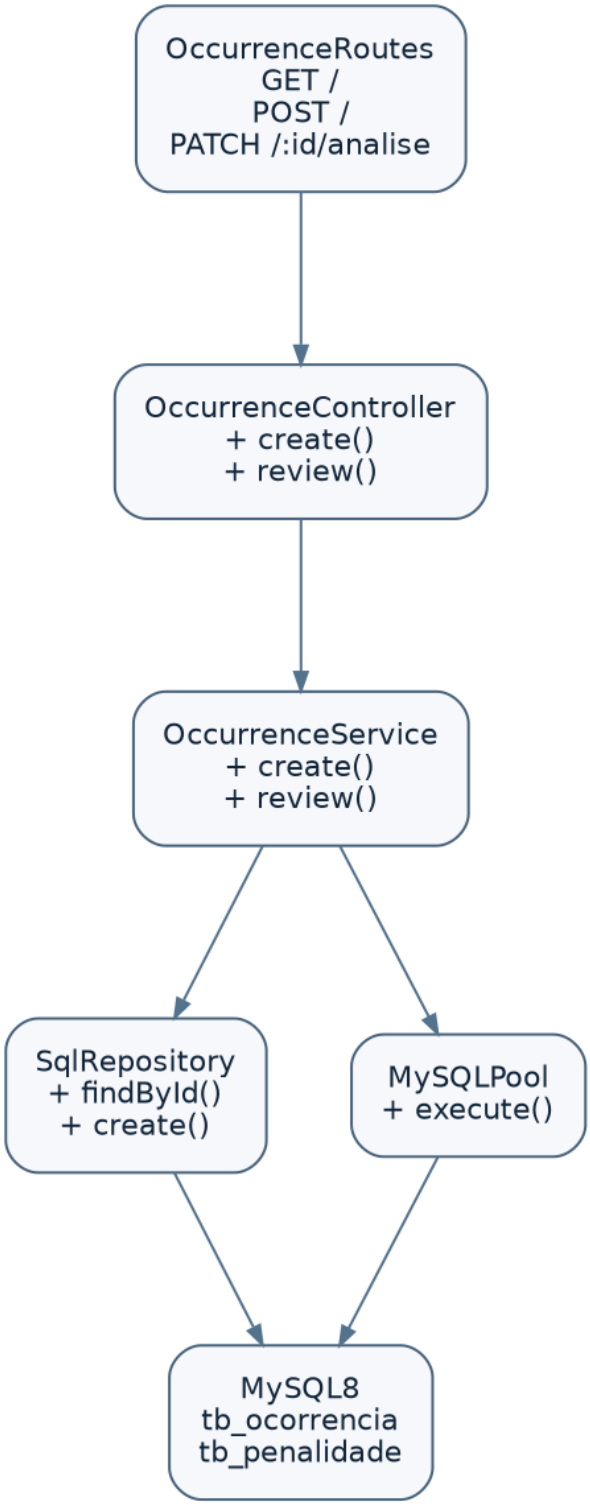
Somente persistência preliminar; o gateway Mercado Pago e a confirmação não estão implementados.



Fonte: diagrama de classes de interação editável no arquivo diagramas/04_reservas_pagamentos.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

11. Diagrama: ocorrências e análise

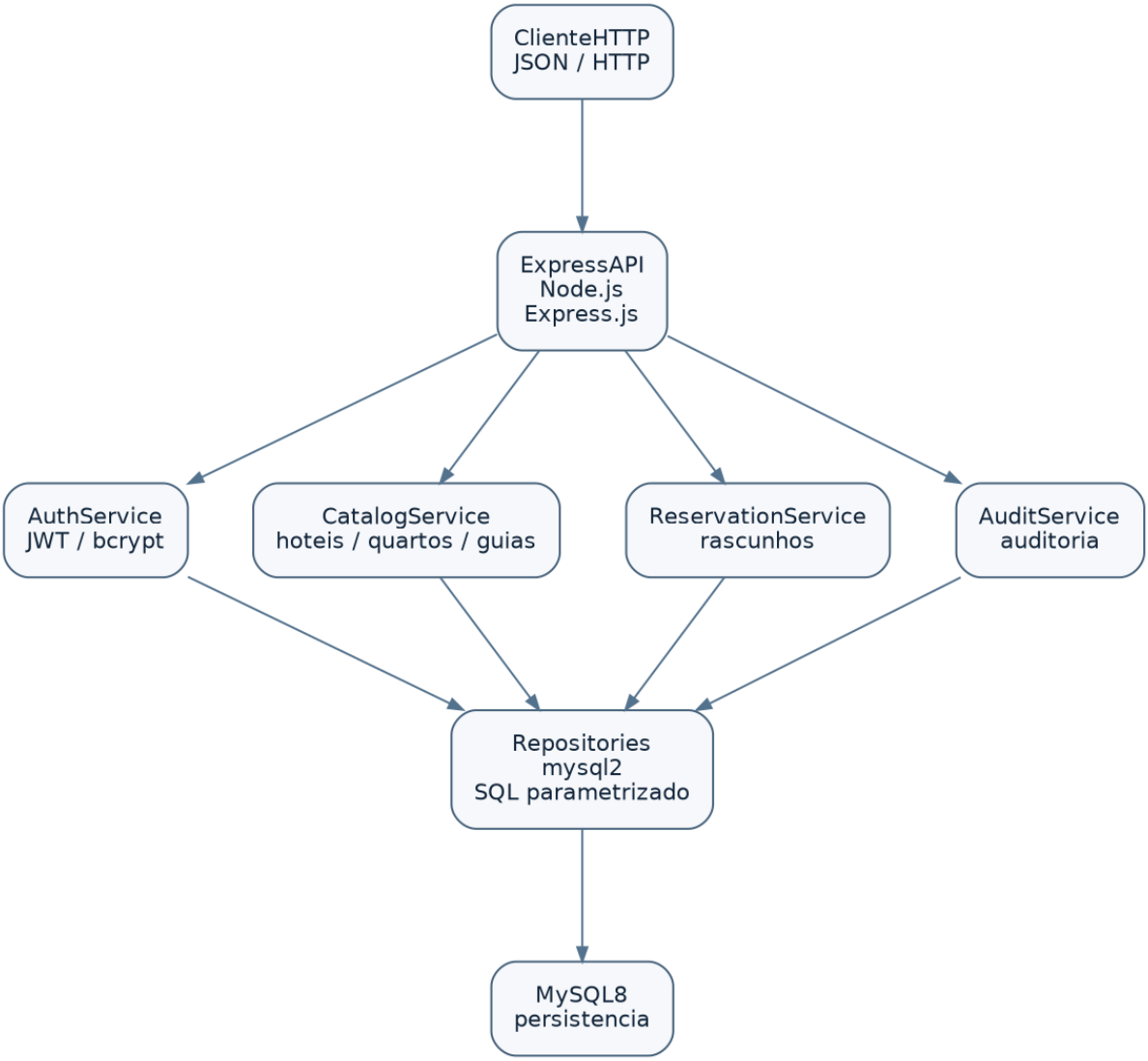
Registro e revisão por administrador, sem pena automatizada.



Fonte: diagrama de classes de interação editável no arquivo diagramas/05_ocorrencias_analise.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

12. Diagrama: componentes atuais

Atualização do desenho de componentes para Node.js/Express + MySQL.



Fonte: diagrama de classes de interação editável no arquivo diagramas/06_componentes_atualizado_node.drawio. Imagem de apresentação gerada a partir do mesmo conteúdo.

13. Matriz-resumo dos requisitos

Distribuição de abrangência estimada com base na presença de código e documentação: 7 implementação inicial; 8 parcial; 20 pendente. Implementação inicial NÃO significa que o funcionamento foi comprovado por testes executados.

RF	Resumo do requisito CP2	Estado
RF01	Permitir cadastro de clientes.	Inicial
RF02	Permitir login e autenticação de usuários.	Inicial
RF03	Permitir pesquisa de hospedagens por destino, datas e quantidade de hóspedes.	Pendente
RF04	Listar hospedagens compatíveis com os critérios de pesquisa.	Parcial
RF05	Exibir detalhes das hospedagens, incluindo informações, comodidades, localização, fotos e va…	Parcial
RF06	Permitir consultar quartos disponíveis e seus detalhes.	Parcial
RF07	Verificar disponibilidade do quarto antes da confirmação da reserva.	Pendente
RF08	Permitir selecionar quarto e calcular o valor da reserva.	Pendente
RF09	Permitir confirmar reservas antecipadas.	Pendente
RF10	Permitir consultar reservas realizadas.	Inicial
RF11	Permitir cancelar reservas conforme as regras definidas.	Parcial
RF12	Integrar o pagamento da reserva ao Mercado Pago.	Pendente
RF13	Registrar e consultar o status do pagamento.	Parcial
RF14	Controlar a regra de pagamento de até 7 dias antes da chegada.	Pendente
RF15	Permitir pesquisar guias.	Parcial
RF16	Permitir consultar guias internos e externos.	Inicial
RF17	Permitir reservar guia interno.	Pendente
RF18	Permitir acessar informações e entrar em contato com guia externo.	Parcial
RF19	Permitir avaliar guias após a utilização do serviço.	Pendente
RF20	Permitir registrar ocorrências relacionadas aos guias.	Inicial
RF21	Permitir ao administrador analisar ocorrências.	Inicial
RF22	Permitir ao administrador aplicar penalidades a guias quando cabível.	Pendente
RF23	Disponibilizar SAC/Chatbot ao cliente autenticado.	Pendente
RF24	Permitir envio de mensagens ao chatbot.	Pendente
RF25	Enviar as mensagens do chatbot à API DeepSeek para processamento.	Pendente
RF26	Apresentar respostas automatizadas geradas pelo serviço de IA.	Pendente
RF27	Permitir encaminhar uma solicitação para atendimento humano quando necessário.	Pendente
RF28	Permitir ao operador de SAC/Ouvidoria visualizar e acompanhar solicitações de atendimento.	Pendente

RF	Resumo do requisito CP2	Estado
RF29	Permitir ao operador de SAC/Ouvidoria registrar interações, encaminhamentos, respostas e enc...	Pendente
RF30	Permitir ao operador de SAC/Ouvidoria consultar informações da reserva e do atendimento nece...	Pendente
RF31	Permitir ao operador de SAC/Ouvidoria sinalizar uma solicitação como escalonada para outro r...	Pendente
RF32	Registrar logs das ações relevantes realizadas no sistema.	Inicial
RF33	Associar os registros de log ao usuário, operação, data e horário correspondentes.	Parcial
RF34	Registrar eventos relevantes das interações com o SAC/Chatbot para rastreabilidade.	Pendente
RF35	Permitir controle do histórico das solicitações de atendimento.	Pendente

14. Divisão de tarefas - cinco integrantes

Responsável	Entregas prioritárias
Integrante 1	MySQL 8: executar schema, seed e verificações; revisar FKs/índices e migrações
Integrante 2	Autenticação, autorização, bcrypt/JWT, privacidade e testes negativos
Integrante 3	Reservas, pagamentos preliminares, regras futuras e testes de integração
Integrante 4	npm/lockfile, Jest, cobertura, CI, Docker e eventual deploy EC2/RDS
Integrante 5	Diagramas, Swagger, PDF final, evidências, SAC/ocorrências e roteiro

Critérios de encerramento do Checkpoint 3

• Jest e MySQL reais funcionando, com capturas de resultados e cobertura.

• Endpoints funcionando com HTTP coerente e privilégios validados.

• Auditoria demonstrada sem logs de senha ou token.

• Swagger navegável, imagens Docker/Compose rodando.

• API pública HTTPS e RDS privado, caso AWS obrigatória no checkpoint.

• Documento e diagramas alinhados, commits reais e aprovação da equipe.

Issues e revisão em grupo

As tarefas, dependências, critérios e exemplos de commits semânticos estão no repositório: docs/DIVISAO_EQUIPE_E_BACKLOG.md. Não há gestão em ProjectLibre/ClickUp neste pacote por escolha da equipe; eventual exigência acadêmica permanece.

15. Roteiro de apresentação - 6 minutos

Tempo	Pessoa	Assunto
0:00–0:45	Integrante 1	Contexto HospedJá e objetivos do checkpoint
0:45–1:50	Integrantes 1–2	Arquitetura Node.js e JWT/bcrypt
1:50–2:55	Integrantes 2–3	Banco, reservas, pagamentos preliminares
2:55–4:00	Integrantes 3–4	TDD didático, testes e Swagger
4:00–4:55	Integrante 4	Docker, logs e AWS (status real)
4:55–5:40	Integrante 5	Ocorrências, matriz de RF e pendências
5:40–6:00	Integrante 5	Próximos passos e encerramento

Perguntas críticas a responder com transparência

• TDD comprovado? Laboratório é didático; novos recursos devem ter histórico real de testes antes do código.

• Pagamentos Mercado Pago? Ainda não integrados; os registros são preliminares.

• Nuvem AWS? Somente roteiro e scripts até que a equipe faça deploy e demonstre URL real.

• Ocorrências geram punição? Não: revisão humana obrigatória, penalidade ainda não exposta em API.

16. Referências e checklist de submissão

Referências internas utilizadas: documento técnico do Checkpoint 2; DER v4.drawio; diagramas de classes/ componentes/ sequência; documento de ocorrências e punições; diretrizes do Checkpoint 3; artefatos das Partes 1, 2 e 3.

Arquivos de apoio no ZIP/repositório

• README.md e docs/RESUMO_EXECUTIVO.md

• database/01_schema.sql, 02_seed.sql, 03_verificacao.sql

• src/ (código da API), src/docs/openapi.yaml, tests/ e tdd/

• diagramas/ (seis fontes .drawio e previews PNG)

• aws/DEPLOY_AWS.md e scripts de implantação

• docs/MATRIZ_REQUISITOS_CP2.md, docs/REVISAO_GERAL.md, docs/TESTES_E_EVIDENCIAS.md

• docs/DIVISAO_EQUIPE_E_BACKLOG.md e docs/ROTEIRO_APRESENTACAO_6_MINUTOS.md

Antes de enviar ao professor

• Preencher identificação completa da equipe, turma, professor e link do GitHub.

• Adicionar somente capturas de execução REAL; omitir senhas e tokens.

• Revisar o relatório de cobertura e confirmar testes e Docker rodando.

• Demonstrar HTTPS/AWS apenas após implantação e validação.

• Conferir se ProjectLibre é exigido apesar da exclusão solicitada pela equipe.

• Unificar evidências adicionais em versão final do PDF único e ensaiar a apresentação.

Observação: o PDF é um documento-base para submissão, ainda pendente de evidências práticas. O repositório concentra as fontes editáveis e código de execução.